Отчёт по лабораторной работе №2

Полина Вячеславовна Белакова

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **8**

4.1 Установка программного обеспечения 8
4.2 Базовая настройка git 9
4.3 Настройка github 10
4.4 Настройка автоматических подписей коммитов git 11
4.5 Настройка gh 11
4.6 Шаблон для рабочего пространства 12
4.7 Настройка каталога курса 12

5 Выводы **14**

6 Контрольные вопросы **15**

Список литературы **18**

Список иллюстраций

4.1 Установка git 8
4.2 Установка gh 8
4.3 Настройка имени и почты владельца репозитория 9
4.4 Настройка git 9
4.5 Создание ключа ssh 9
4.6 Создание ключа pgp 10
4.7 Добавление ключа pgp 10
4.8 Копирование ключа pgp 10
4.9 Настраиваю автоматическую подпись коммитов 11
4.10 Авторизация 11
4.11 Создание директории 12
4.12 Создание репозиторя и его клонирование 12

Список таблиц

1 Цель работы

Изучить идеологию и применение средств контроля версий и приобрести навыки работы с git.

2 Задание

- Изучить идеологию и применение средств контроля версий.
- Освоить умения по работе с git.

3 Теоретическое введение

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется. Среди классических VCS наиболее известны CVS, Subversion, а среди распределённых — Git, Bazaar, Mercurial. Принципы их работы схожи, отличаются они в основном синтаксисом используемых в работе команд. Система контроля версий Git представляет собой набор программ командной строки. Доступ к ним можно получить из терминала посредством ввода команды git с различными опциями. Благодаря тому, что Git является распределённой системой контроля версий, резервную копию локального хранилища можно сделать простым копированием или архивацией.

4 Выполнение лабораторной работы

4.1 Установка программного обеспечения

Установила git(рис. 4.1).

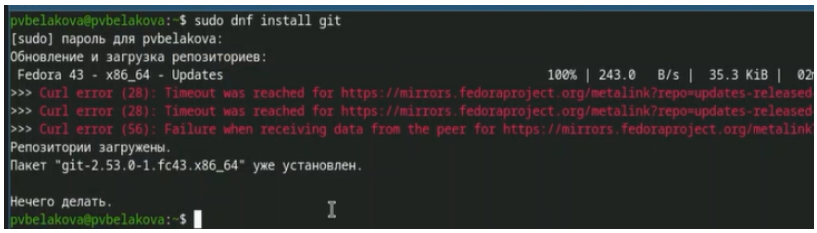


Рисунок 4.1: Установка git

Установила gh (рис. 4.2).

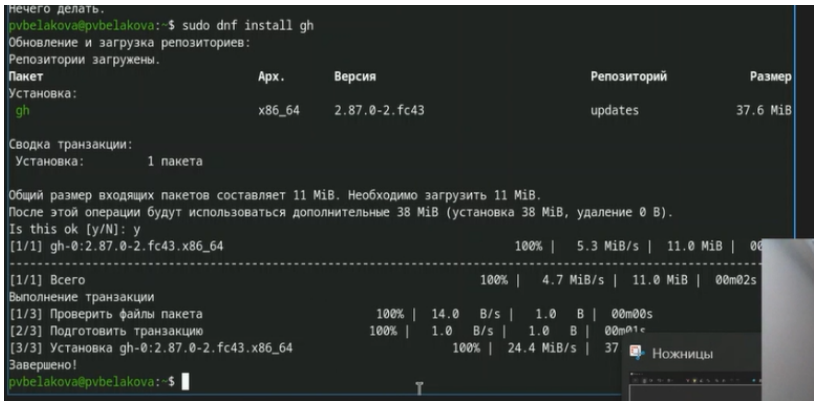


Рисунок 4.2: Установка gh

4.2 Базовая настройка git

Зададаю имя и email владельца репозитория (рис. 4.3).

```
Завершено!
pvbelakova@pvbelakova:~$ git config --global user.name "Polina Belakova"
pvbelakova@pvbelakova:~$ git config --global user.email "belakovapolina@mail.ru"
```

Рисунок 4.3: Настройка имени и почты владельца репозитория

Настраиваю utf-8 в выводе сообщений, верификацию и подписание коммитов git, зададаю имя начальной ветки, настраиваю параметры autocrlf, safecrlf Установила gh (рис. 4.4).

```
pvbelakova@pvbelakova:~$ git config --global core.quotepath false
pvbelakova@pvbelakova:~$ git config --global init.defaultBranch master
pvbelakova@pvbelakova:~$ git config --global core.autocrlt input
pvbelakova@pvbelakova:~$ git config --global core.safecrlf warn
pvbelakova@pvbelakova:~$ git config --global core.autocrlf input
```

Рисунок 4.4: Настройка git

Создаю ключи ssh (рис. 4.5).

```
pvbelakova@pvbelakova:~$ ssh-keygen -t rsa -b 4096 -C "belakovapolina@mail.ru"
Generating public/private rsa key pair.
Enter file in which to save the key (/home/pvbelakova/.ssh/id_rsa):
Created directory '/home/pvbelakova/.ssh'.
Enter passphrase for "/home/pvbelakova/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/pvbelakova/.ssh/id_rsa
Your public key has been saved in /home/pvbelakova/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:uVxdPRlcDByqlWsJWSYieBynDOITo1s1R/7jv/nXQlM belakovapolina@mail.ru
The key's randomart image is:
+---[RSA 4096]----+
|. o ++=.o . o.+=o|
|.o +.*o+ . = o.o+|
|. +  .+   o + .o.|
| o .   . . = +  E|
|.       S o =  . |
|       o + .  o  |
|        +    . o |
|         . .  o .|
|          +o.. . |
+----[SHA256]-----+
pvbelakova@pvbelakova:~$
```

Рисунок 4.5: Создание ключа ssh

Создаю ключи pgp(рис. 4.6).

```
pvbelakova@pvbelakova:~$ gpg --full-generate-key
gpg (GnuPG) 2.4.8; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: создан каталог '/home/pvbelakova/.gnupg'
Выберите тип ключа:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (только для подписи)
  (14) Existing key from card
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
Запрошенный размер ключа - 4096 бит
Выберите срок действия ключа.
```

Рисунок 4.6: Создание ключа pgp

4.3 Настройка github

Добавляю PGP ключ в GitHub(рис. 4.7).

```
pvbelakova@pvbelakova:~$ gpg --list-secret-key --keyid-format LONG
gpg: проверка таблицы доверия
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: глубина: 0  достоверных:   1  подписанных:   0  доверие: 0-, 0q, 0n, 0m, 0f, 1u
[keyboxd]
---------
sec   rsa4096/BF6A0E0EAC633743 2026-03-05 [SC]
      2E3ACE4A4F49DF75ABEA396FBF6A0E0EAC633743
uid                 [  абсолютно ] Polina Belakova <belakovapolina@mail.ru>
ssb   rsa4096/99AA5753ED980273 2026-03-05 [E]
```

Рисунок 4.7: Добавление ключа pgp

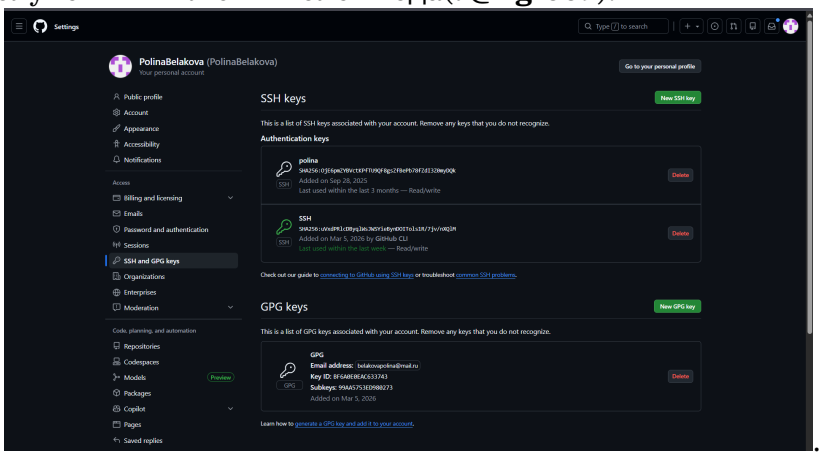
Копирую сгенерированный PGP ключ в буфер обмена (рис. 4.8).

```
pvbelakova@pvbelakova:~$ gpg --armor --export BF6A0E0EAC633743
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQINBGmpquIBEADD0YNpqJT8TSkPWwWHLVAVQa0uTdnLbE+ThWqF2SNVvWqKmxaQ
9GHrvpNBHRNCfJp8vISDWS1N99JZjvRNbCnAUtpVglMbMXl99UqL0T+8ksJ6jIun
6v4AeunC5GCG2D2JVpqamIaxTBkwPlkve+xMMEaP/cxC8foB9RZC/tMMOsmDGEf2
zmAxl4/0KQORH8hnQnhatl0Tn0/EjW9YvY/00PPO6563CC+LzNa7+39EiqWNqnXN
587snqzxTbYmJLL4L5f71xzPqw++ih9Wf5NhvwyFkrMvav+RBo3TrnrhJt6nuBF/
0az7BtV8J5bKipv2AosHnFSiaEHmT7lel3Ez1pRCc5ZduXLG6zw5O5umDbi+v1ff
```

Рисунок 4.8: Копирование ключа pgp

Перехожу в настройки GitHub, нажимаю на кнопку New GPG key и вставьте полученный ключ в поле ввода(**?@fig-009**).



4.4 Настройка автоматических подписей коммитов git

Используя введёный email, указываю примениние Git при подписи коммитов (рис. 4.9).

```
pvbelakova@pvbelakova:~$ git config --global user.singinkey BF6A0E0EAC633743
pvbelakova@pvbelakova:~$ git config --global commit.gpgsing true
pvbelakova@pvbelakova:~$ git config --global gpg.program $(which gpg)
```

Рисунок 4.9: Настраиваю автоматическую подпись коммитов

4.5 Настройка gh

Авторизоввываюсь с помощью команды gh auth login (рис. 4.10).

```
pvbelakova@pvbelakova:~/work/study/2025-2026/Операционные системы$ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? SSH
? Upload your SSH public key to your GitHub account? /home/pvbelakova/.ssh/id_rsa.pub
? Title for your SSH key: SSH
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 4E6B-2105
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol ssh
✓ Configured git protocol
✓ Uploaded the SSH key to your GitHub account: /home/pvbelakova/.ssh/id_rsa.pub
✓ Logged in as PolinaBelakova
```

Рисунок 4.10: Авторизация

4.6 Шаблон для рабочего пространства

Создаю дерикторию (рис. 4.11)и репозиторий курса на основе шаблона(рис. 4.12).

```
pvbelakova@pvbelakova:~$ mkdir -p ~/work/study/2025-2026/"Операционные системы"
```

Рисунок 4.11: Создание директории

```
pvbelakova@pvbelakova:~/work/study/2025-2026/Операционные системы$ gh repo create study_2025-2026_os-
ntro --template=yamadharma/course-directory-student-template --public
✓ Created repository PolinaBelakova/study_2025-2026_os-intro on github.com
  https://github.com/PolinaBelakova/study_2025-2026_os-intro
pvbelakova@pvbelakova:~/work/study/2025-2026/Операционные системы$ git clone --recursive git@github.c
m:PolinaBelakova/study_2025-2026_os-intro.git os-intro
Клонирование в «os-intro»...
remote: Enumerating objects: 41, done.
remote: Counting objects: 100% (41/41), done.
remote: Compressing objects: 100% (39/39), done.
Получение объектов: 100% (41/41), 25.54 KiB | 25.54 MiB/s, готово.
Определение изменений: 100% (1/1), готово.
remote: Total 41 (delta 1), reused 38 (delta 1), pack-reused 0 (from 0)
```

Рисунок 4.12: Создание репозиторя и его клонирование

4.7 Настройка каталога курса

Перехожу в каталог курса, удаляю лишние файлы (**?@fig-014**), создаю необходимые каталоги (**?@fig-015**)

```
pvbelakova@pvbelakova:~/work/study/2025-2026/Операционные системы$ cd os-intro
pvbelakova@pvbelakova:~/work/study/2025-2026/Операционные системы/os-intro$ ls
CHANGELOG.md        CONTRIBUTING.md  LICENSE   package.json  README.git-flow.md  SECURITY.md
CODE_OF_CONDUCT.md  COURSE           Makefile  README.en.md  README.md           template
pvbelakova@pvbelakova:~/work/study/2025-2026/Операционные системы/os-intro$ rm package.json
pvbelakova@pvbelakova:~/work/study/2025-2026/Операционные системы/os-intro$ ls
CHANGELOG.md        CONTRIBUTING.md  LICENSE   README.en.md        README.md    template
CODE_OF_CONDUCT.md  COURSE           Makefile  README.git-flow.md  SECURITY.md
```

```
pvbelakova@pvbelakova:~/work/study/2025-2026/Операционные системы
pvbelakova@pvbelakova:~/work/study/2025-2026/Операционные системы
Synchronizing submodule url for 'template/report'
Synchronizing submodule url for 'template/presentation'
Synchronizing submodule url for 'template/presentation'
Synchronizing submodule url for 'template/report'
sed: невозможно прочитать package.json: Нет такого файла или ката
sed: невозможно прочитать package.json: Нет такого файла или ката
sed: невозможно прочитать package.json: Нет такого файла или ката
sed: невозможно прочитать package.json: Нет такого файла или ката
sed: невозможно прочитать package.json: Нет такого файла или ката
make: *** [Makefile:22: prepare] Ошибка 2
```

Отправляю файлы на сервер (**?@fig-016**), (**?@fig-017**).

```
pvbelakova@pvbelakova:~/work/study/2025-2026/Операционные системы/os-intro$ git add .
pvbelakova@pvbelakova:~/work/study/2025-2026/Операционные системы/os-intro$ git commit -am "создание
труктуры курса"
[master aabf050] создание структуры курса
 525 files changed, 25896 insertions(+), 254 deletions(-)
 delete mode 100644 CHANGELOG.md
 create mode 100644 labs/README.md
 create mode 100644 labs/README.ru.md
 create mode 100644 labs/lab01/presentation/.gitignore
 create mode 100644 labs/lab01/presentation/.marksman.toml
 create mode 100644 labs/lab01/presentation/.projectile
 create mode 100644 labs/lab01/presentation/Makefile
 create mode 100644 labs/lab01/presentation/_quarto.yml
 create mode 100644 labs/lab01/presentation/_resources/image/logo_rudn.png
 create mode 100644 labs/lab01/presentation/_resources/tex/beamer.tex
 create mode 100644 labs/lab01/presentation/image/kulyabov.jpg
 create mode 100644 labs/lab01/presentation/os-intro-lab01-os-intro-lab01-presentation.qmd
 create mode 100644 labs/lab01/presentation/os-intro-lab01-presentation.qmd
 create mode 100644 labs/lab01/report/.gitignore
 create mode 100644 labs/lab01/report/.marksman.toml
 create mode 100644 labs/lab01/report/.projectile
 create mode 100644 labs/lab01/report/Makefile
 create mode 100644 labs/lab01/report/_extensions/yamadharma/minted-quarto/_extension.yml
 create mode 100644 labs/lab01/report/_extensions/yamadharma/minted-quarto/minted-quarto.lua
 create mode 100644 labs/lab01/report/_quarto.yml
```

```
pvbelakova@pvbelakova:~/work/study/2025-2026/Операционны
Перечисление объектов: 105, готово.
Подсчет объектов: 100% (105/105), готово.
При сжатии изменений используется до 4 потоков
Сжатие объектов: 100% (89/89), готово.
Запись объектов: 100% (104/104), 705.08 KiB | 5.26 MiB/s
Total 104 (delta 41), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (41/41), completed with 1
To github.com:PolinaBelakova/study_2025-2026_os-intro.gi
   c188b4a..aabf050  master -> master
```

5 Выводы

Была изучена идеология и применение средств контроля версий.Приобретены навыки работы с git.

6 Контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются? Система контроля версий (VCS) — это программное обеспечение для отслеживания изменений в файлах и координации работы над ними. VCS хранит историю изменений, позволяет возвращаться к предыдущим версиям и определять авторство каждой правки. Главные задачи: обеспечение совместной работы нескольких человек над проектом, защита от потери данных и возможность экспериментировать без риска сломать основной код.

2. Объясните понятия : хранилище, commit, история, рабочая копия. Хранилище (repository) — это место, где VCS хранит все файлы проекта и всю историю их изменений. Commit — операция сохранения текущего состояния файлов в хранилище, «снимок» проекта в определённый момент времени. История — цепочка последовательных commit'ов, позволяющая проследить эволюцию проекта. Рабочая копия — текущая версия файлов на вашем компьютере, с которой вы непосредственно работаете.

3. Чем отличаются централизованные и децентрализованные VCS? В централизованных VCS (например, SVN) существует одно главное хранилище на сервере, а разработчики получают только рабочие копии; при недоступности сервера работа с историей невозможна. В децентрализованных VCS (например, Git) каждый разработчик имеет полную копию хранилища с историей на своём компьютере, что позволяет работать

автономно. Любая локальная копия в DVCS является полноценным резервным хранилищем проекта.

4. Опишите действия с VCS при единоличной работе. При единоличной работе сначала инициализируется пустое хранилище в папке проекта (git init). Затем создаются и изменяются файлы, после чего они добавляются в индекс (git add) и фиксируются (git commit) с пояснительным сообщением. В любой момент можно посмотреть историю изменений (git log) или вернуться к более ранней версии проекта.

5. Опишите порядок работы с общим хранилищем VCS. Работа начинается с клонирования удалённого репозитория (git clone). Перед началом работы и перед отправкой изменений необходимо загрузить актуальную версию от коллег (git pull). После внесения изменений они фиксируются локально (git commit), а затем отправляются в общее хранилище (git push). При возникновении конфликтов их нужно разрешить вручную и сделать commit слияния.

6. Каковы основные задачи git? Git обеспечивает полную историю изменений проекта с возможностью навигации по ней. Он поддерживает нелинейную разработку через мощную систему ветвления и гарантирует целостность данных с помощью контрольных сумм. Git позволяет каждому разработчику иметь полную копию репозитория и работать автономно, а также предоставляет гибкие инструменты для управления версиями (теги, ветки).

7. Назовите и дайте краткую характеристику командам git. git init создаёт новый локальный репозиторий, а git clone копирует удалённый. git add добавляет изменения в индекс, git commit фиксирует их в репозитории. git push отправляет изменения на удалённый сервер, а git pull загружает их оттуда. git branch и git checkout управляют ветками, git merge сливает

ветки, а git status и git log показывают состояние и историю.

8. Приведите примеры использования с локальным и удалённым репозиториями. Локально git init → создаём файлы → git add → git commit -m ’сообщение” → git log для просмотра истории. С удалённым: git clone → git pull для обновления → работаем → git commit → git push для отправки изменений. Для новой задачи создаём ветку git checkout -b feature, а после завершения отправляем её git push origin feature.

9. Что такое ветви (branches) и зачем они нужны? Ветка — это перемещаемый указатель на один из commit’ов, позволяющий вести разработку изолированно от основной линии. Ветки нужны для параллельной разработки новых функций без риска сломать стабильный код. Они также используются для срочного исправления ошибок, проведения экспериментов и организации работы нескольких разработчиков над одним проектом.

10. Как и зачем игнорировать файлы при commit? Для игнорирования файлов используется файл .gitignore, в котором перечисляются шаблоны имён файлов и папок, не подлежащих версионированию. Игнорируют временные файлы редакторов, скомпилированные бинарные файлы, конфиденциальные данные и папки с зависимостями. Это позволяет держать репозиторий чистым и лёгким, содержащим только исходный код и документацию.

Список литературы